Formalization of Constrained and Metric Flow Matching v1

Project Notes

April 29, 2026

1 Goal and Setting

We want to learn a velocity field that transports p_0 to p_1, while enforcing intermediate-time moment constraints. In v1, the state is $x \in \mathbb{R}^2$.

Given

- Initial and terminal marginals p_0, p_1.
- A coupling over endpoint pairs (x_0, x_1): batch OT (`ot`), random permutation (`random`), or cached global balanced OT plan sampling (`ot_global`).
- Constraint times $\mathcal{T}_c = \{0.25, 0.5, 0.75\}$.
- Target moments m_k at each $t_k \in \mathcal{T}_c$.

Wanted Learn:

- a path-correction model $g_\theta(t, x_0, x_1)$,
- a velocity model $v_\psi(t, x)$,

such that induced paths satisfy moment constraints and v_ψ matches path velocities.

2 Reference: Standard CFM

Standard conditional flow matching uses a fixed interpolation path x_t (often linear) and target velocity $u_t = \partial_t x_t$, then trains

$$\mathcal{L}_{\mathrm{CFM}}(\psi) = \mathbb{E}_{(x_0,x_1),\,t}\left[\|v_\psi(t, x_t) - u_t\|^2\right].$$

3 Path Family Used in This Project

3.1 Baseline Path

$$\ell_t(x_0, x_1) = (1-t)x_0 + tx_1.$$

3.2 Constrained-Correction Path

We use an endpoint-preserving correction:

$$x_t = \ell_t + a(t)\, g_\theta(t, x_0, x_1), \qquad a(t) = t(1-t).$$

Because $a(0) = a(1) = 0$, endpoints are exactly preserved: $x_{t=0} = x_0$, $x_{t=1} = x_1$.

3.3 Target Velocity for CFM in Constrained Mode

$$u_t = \partial_t x_t = (x_1 - x_0) + (1-2t)\, g_\theta(t, x_0, x_1) + a(t)\, \partial_t g_\theta(t, x_0, x_1).$$

This is exactly what is computed in code using autograd for $\partial_t g_\theta$.

4 Moment Constraints

4.1 Feature Map Used in v1

For a batch $\{x^{(i)}\}_{i=1}^B \subset \mathbb{R}^2$:

$$\hat{\mu} = \frac{1}{B}\sum_{i=1}^{B} x^{(i)}, \qquad \hat{\Sigma} = \frac{1}{B}\sum_{i=1}^{B} \big(x^{(i)} - \hat{\mu}\big)\big(x^{(i)} - \hat{\mu}\big)^\top.$$

Define

$$\phi(\{x^{(i)}\}_{i=1}^B) = \begin{bmatrix} \hat{\mu} \\ \mathrm{vec}(\hat{\Sigma}) \end{bmatrix} \in \mathbb{R}^6.$$

4.2 Residual at Constraint Times

For each $t_k \in \mathcal{T}_c$:

$$c_k(\theta) = \phi\Big(\{x_{t_k}^{(i)}(\theta)\}_{i=1}^B\Big) - m_k.$$

Small $\|c_k\|$ means constraints are well satisfied.

5 How Target Moments m_k are Built in v1

In the benchmark, (p_0, p_1) are Gaussians:

$$p_0 = \mathcal{N}(\mu_0, \Sigma_0), \qquad p_1 = \mathcal{N}(\mu_1, \Sigma_1).$$

Define the Gaussian OT map matrix:

$$A = \Sigma_0^{-1/2}\left(\Sigma_0^{1/2}\Sigma_1\Sigma_0^{1/2}\right)^{1/2}\Sigma_0^{-1/2}.$$

Define

$$B_t = (1-t)I + tA, \qquad s(t) = 1 + \kappa\, t(1-t).$$

Then target mean/covariance are:

$$\mu_t = (1-t)\mu_0 + t\mu_1, \qquad \Sigma_t = s(t)^2\, B_t \Sigma_0 B_t^\top.$$

Finally:

$$m_k = \begin{bmatrix} \mu_{t_k} \\ \mathrm{vec}(\Sigma_{t_k}) \end{bmatrix}.$$

5.1 Bridge benchmark target construction

For the bridge benchmark, target constraints are not analytic Gaussians. Instead, we simulate a reference bridge SDE and estimate targets from high-sample Monte Carlo snapshots:

$$\{x_{t_k}^{(i)}\}_{i=1}^{N_{\text{target}}}, \qquad N_{\text{target}} = 200000 \text{ (default)}.$$

Bridge times are represented in normalized coordinates $t_{\text{norm}} \in [0, 1]$, and mapped to physical SDE time by

$$t_{\text{phys}} = t_{\text{norm}} \cdot T_{\text{bridge}},$$

where $T_{\text{bridge}} =$ `bridge.total_time` (now set to 1.5 in bridge experiment configs). So constraints at normalized times $\{0.25, 0.5, 0.75\}$ are matched to physical snapshots at $\{0.375, 0.75, 1.125\}$, and endpoint $t_{\text{norm}} = 1$ uses physical time 1.5. For each constrained time t_k:

$$m_k = \begin{bmatrix} \hat{\mu}_{t_k} \\ \text{vec}(\hat{\Sigma}_{t_k}) \end{bmatrix},$$

where $(\hat{\mu}_{t_k}, \hat{\Sigma}_{t_k})$ are empirical moments of the cached bridge samples. These high-precision targets are cached under `.cache/bridge_targets/` keyed by bridge data config, seed, and constrained times, then reused across runs.

6 Loss Terms Used in the Current Implementation

6.1 Path Regularizer

$$R(\theta) = \alpha\, \mathbb{E}\big[\|u_t - (x_1 - x_0)\|^2\big] + \mathbb{E}\big[\beta(t)\, \|\partial_t u_t\|^2\big]\,.$$

Interpretation

- First term keeps the learned path velocity near linear transport velocity.
- Second term penalizes rapid temporal variation of velocity (temporal smoothness).

6.2 Time-Varying Smoothness Weighting in Stage A/C

For constrained mode, Stage A and Stage C now use a time-varying smoothness weight $\beta(t)$ (with backward-compatible constant mode). Let the anchor set be

$$\mathcal{A} = \{0 = t_0 < t_1 < \cdots < t_M = 1\},$$

where anchors are endpoints plus constrained times. Define full moment feature vectors

$$m_t = \begin{bmatrix} \mu_t \\ \text{vec}(\Sigma_t) \end{bmatrix},$$

and interval drift magnitudes

$$d_i = \frac{\|m_{t_{i+1}} - m_{t_i}\|_2}{t_{i+1} - t_i}, \qquad i = 0, \ldots, M-1.$$

With $\bar{d} = \frac{1}{M}\sum_{i=0}^{M-1} d_i$, base weight β_{base}, and scheduler hyperparameters $(p, \epsilon, s_{\min}, s_{\max})$, the interval scales are

$$s_i = \text{clip}\left(\left(\frac{\bar{d}}{d_i + \epsilon}\right)^p,\ s_{\min},\ s_{\max}\right), \qquad \beta_i = \beta_{\text{base}}\, s_i.$$

Piecewise schedule

$$\beta(t) = \beta_i, \qquad t \in [t_i, t_{i+1}).$$

Linear schedule First define anchor weights from adjacent intervals:

$$\tilde{\beta}_0 = \beta_0, \qquad \tilde{\beta}_M = \beta_{M-1}, \qquad \tilde{\beta}_i = \frac{\beta_{i-1} + \beta_i}{2} \quad (i = 1, \ldots, M-1).$$

Then for $t \in [t_i, t_{i+1}]$,

$$\beta(t) = \tilde{\beta}_i + \frac{t - t_i}{t_{i+1} - t_i}\big(\tilde{\beta}_{i+1} - \tilde{\beta}_i\big).$$

Constant schedule (backward compatibility)

$$\beta(t) \equiv \beta_{\text{base}}.$$

Updated regularizer form used in code

$$R(\theta) = \alpha \cdot \frac{1}{B} \sum_{i=1}^{B} \|u_{t^{(i)}} - (x_1^{(i)} - x_0^{(i)})\|_2^2 + \frac{1}{B} \sum_{i=1}^{B} \beta(t^{(i)}) \, \|\partial_t u_{t^{(i)}}\|_2^2.$$

6.3 Augmented Lagrangian for Path Learning (Stage A/C)

$$\mathcal{L}_g(\theta, \lambda) = R(\theta) + \sum_k \lambda_k^\top c_k(\theta) + \frac{\rho}{2} \sum_k \|c_k(\theta)\|^2.$$

Multiplier updates (with clipping in code):

$$\lambda_k \leftarrow \text{clip}(\lambda_k + \rho \, c_k(\theta), \, -\lambda_{\max}, \, \lambda_{\max}) \, .$$

Algorithm 1: Mini-batch augmented-Lagrangian updates (code-equivalent)

Inputs: constraint times $\mathcal{T}_c$, target features $\{m_k\}_{t_k \in \mathcal{T}_c}$, hyperparameters $(\alpha, \beta_{\text{base}}, \rho, \eta_{\text{joint}}, \lambda_{\max}, p, \epsilon, s_{\min}, s_{\max})$, schedule type (‘constant‘, ‘piecewise‘, or ‘linear‘), and step counts (N_A, N_B, N_C).

1. Initialize multipliers: $\lambda_k \leftarrow 0$ for all $t_k \in \mathcal{T}_c$.
2. **Stage A (optimize θ only):** for $s = 1, \ldots, N_A$:
   (a) Sample paired mini-batch $\{(x_0^{(i)}, x_1^{(i)})\}_{i=1}^B$.
   (b) Sample $t^{(i)} \sim \text{Uniform}(0, 1)$ and compute constrained velocity targets $u_{t^{(i)}}(\theta)$.
   (c) Compute regularizer

$$R(\theta) \leftarrow \alpha \, \frac{1}{B} \sum_i \|u_{t^{(i)}} - (x_1^{(i)} - x_0^{(i)})\|_2^2 + \frac{1}{B} \sum_i \beta(t^{(i)}) \, \|\partial_t u_{t^{(i)}}\|_2^2.$$

   (d) For each $t_k \in \mathcal{T}_c$, compute

$$x_{t_k}^{(i)} \leftarrow (1 - t_k) x_0^{(i)} + t_k x_1^{(i)} + t_k(1 - t_k) \, g_\theta(t_k, x_0^{(i)}, x_1^{(i)}),$$

$$\hat{\mu}_k \leftarrow \frac{1}{B} \sum_i x_{t_k}^{(i)}, \qquad \hat{\Sigma}_k \leftarrow \frac{1}{B} \sum_i (x_{t_k}^{(i)} - \hat{\mu}_k)(x_{t_k}^{(i)} - \hat{\mu}_k)^\top,$$

$$c_k(\theta) \leftarrow \begin{bmatrix} \hat{\mu}_k \\ \text{vec}(\hat{\Sigma}_k) \end{bmatrix} - m_k.$$

(e) Assemble AL objective

$$\mathcal{L}_g \leftarrow R(\theta) + \sum_k \lambda_k^\top c_k(\theta) + \frac{\rho}{2} \sum_k \|c_k(\theta)\|_2^2.$$

(f) Apply Adam step on θ with gradient $\nabla_\theta \mathcal{L}_g$.

(g) Update multipliers for all $t_k \in \mathcal{T}_c$:

$$\lambda_k \leftarrow \mathrm{clip}(\lambda_k + \rho\, c_k(\theta), -\lambda_{\max}, \lambda_{\max})\,.$$

3. **Stage B (freeze θ, optimize ψ):** for $s = 1, \ldots, N_B$, sample a mini-batch and apply Adam step on ψ with $\nabla_\psi \mathcal{L}_{\mathrm{CFM}}$.

4. **Stage C (joint optimize (ψ, θ)):** for $s = 1, \ldots, N_C$:

   (a) Sample a mini-batch and recompute $\mathcal{L}_g(\theta, \lambda)$ exactly as in Stage A.

   (b) Compute $\mathcal{L}_{\mathrm{CFM}}(\psi; \theta)$ and

   $$\mathcal{L}_{\mathrm{joint}} \leftarrow \mathcal{L}_{\mathrm{CFM}} + \eta_{\mathrm{joint}} \mathcal{L}_g.$$

   (c) Apply Adam step on (ψ, θ) with gradient $\nabla_{(\psi,\theta)} \mathcal{L}_{\mathrm{joint}}$.

   (d) Update all multipliers with the same clipped rule:

   $$\lambda_k \leftarrow \mathrm{clip}(\lambda_k + \rho\, c_k(\theta), -\lambda_{\max}, \lambda_{\max})\,.$$

6.4 CFM Loss for Velocity Network

$$\mathcal{L}_{\mathrm{CFM}}(\psi; \theta) = \mathbb{E}_{(x_0, x_1), t} \left[\|v_\psi(t, x_t(\theta)) - u_t(\theta)\|^2 \right].$$

6.5 Joint Finetune Objective (Stage C)

$$\mathcal{L}_{\mathrm{joint}}(\psi, \theta, \lambda) = \mathcal{L}_{\mathrm{CFM}}(\psi; \theta) + \eta_{\mathrm{joint}}\, \mathcal{L}_g(\theta, \lambda).$$

7 Current Optimization Loop (Exactly What Code Does)

Sampling per Step

At each step, endpoint pairing depends on configured coupling:

1. `ot`: sample endpoint mini-batches from p_0 and p_1, then solve exact one-to-one assignment (Hungarian) on that batch.

2. `random`: sample endpoint mini-batches from p_0 and p_1, then apply a random permutation pairing.

3. `ot_global` (single-cell): precompute one balanced global Kantorovich plan

   $$\Pi \in \mathbb{R}_+^{N_0 \times N_1},$$

   once per run setup (cacheable), store sparse support

   $$\mathcal{E} = \{(i, j, \Pi_{ij}) : \Pi_{ij} > 0\},$$

   then sample paired indices $(i, j) \sim \Pi$ *with replacement* for each SGD step.

4. Sample $t \sim \mathrm{Uniform}(0, 1)$ for CFM/regularizer terms.

For `ot_global`, pair sampling is stochastic from the fixed plan support (not deterministic replay of one static matching list).

Stage A: Pretrain Path Model g_θ

For N_A steps:

1. Compute $\mathcal{L}_g(\theta, \lambda)$.
2. Gradient step on θ (Adam, lr_g).
3. Update $\lambda_k \leftarrow \lambda_k + \rho c_k$ (with clipping).

Stage B: Train Velocity Model v_ψ with Frozen g_θ

For N_B steps:

1. Freeze θ.
2. Compute $\mathcal{L}_{\text{CFM}}(\psi; \theta)$.
3. Gradient step on ψ (Adam, lr_v).

Stage C: Joint Finetune

For N_C steps:

1. Unfreeze both θ, ψ.
2. Compute $\mathcal{L}_{\text{joint}}$.
3. Gradient step on (θ, ψ).
4. Update multipliers λ_k again.

Baseline Mode

In baseline mode there is no g_θ:

$$x_t = \ell_t, \qquad u_t = x_1 - x_0.$$

Only Stage B is optimized.

8 Constrained-Information Metric Flow Matching (Endpoint LAND + Moment Constraints)

This section formalizes the new hybrid metric variants:

`metric_constrained_al`, `metric_constrained_soft`.

The design goal is a fair comparison: LAND geometry is learned from endpoint marginals only, while intermediate information is injected only through moment constraints.

MFM mean path and conditional flow

Let $g_\theta(t, x_0, x_1)$ be the geopath network and $\alpha_{\text{mfm}} \geq 0$. On $[0, 1]$, define

$$\gamma(t) = 1 - t^2 - (1-t)^2 = 2t(1-t), \qquad \dot{\gamma}(t) = 2(1-2t).$$

The MFM mean path is

$$\mu_t = (1-t)x_0 + tx_1 \;+\; \alpha_{\text{mfm}}\gamma(t)\, g_\theta(t, x_0, x_1).$$

Its time derivative is

$$u_t = \partial_t \mu_t = (x_1 - x_0)\alpha_{\text{mfm}} \left[\dot{\gamma}(t)\, g_\theta(t, x_0, x_1)\gamma(t)\, \partial_t g_\theta(t, x_0, x_1)\right].$$

Noisy state used in Stage B metric flow matching

Stage B uses noisy locations

$$x_t = \mu_t + \sigma_t \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I),$$

with $\sigma_t = \sigma$ in the current native backend implementation. The velocity target remains u_t. Therefore the Stage B objective is unchanged:

$$\mathcal{L}_{\text{CFM}}^{\text{metric}}(\psi; \theta) = \mathbb{E}_{(x_0, x_1), t, \varepsilon} \left\| v_\psi(t, x_t) - u_t \right\|_2^2.$$

LAND metric tensor and geodesic loss

Given query points $x \in \mathbb{R}^{B \times d}$ and manifold samples $\{z_n\}_{n=1}^N \subset \mathbb{R}^d$, define RBF weights

$$w_n(x) = \exp\left(-\frac{\|x - z_n\|_2^2}{2\gamma_{\text{land}}^2}\right),$$

and diagonal inverse metric entries

$$\left[G^{-1}(x)\right]_{jj} = \frac{1}{\sum_{n=1}^N w_n(x)\,(z_{n,j} - x_j)^2 \rho_{\text{land}}}.$$

The LAND geodesic loss is

$$\mathcal{L}_{\text{LAND}}(\theta) = \mathbb{E}\left[u_t^\top G^{-1}(\mu_t) u_t\right] = \mathbb{E}\left[\sum_{j=1}^d u_{t,j}^2 \left[G^{-1}(\mu_t)\right]_{jj}\right].$$

Endpoint-only manifold policy (fairness default)

Define endpoint manifold pool

$$\mathcal{M}_{\text{end}} = \{x_0^{(i)}\}_{i=1}^{N_0} \cup \{x_1^{(j)}\}_{j=1}^{N_1}.$$

In default policy `mfm.reference_pool_policy=endpoints_only`, $\mathcal{L}_{\text{LAND}}$ uses only samples from $\mathcal{M}_{\text{end}}$. Intermediate marginals are excluded from LAND metric construction. (Optional ablation `anchors_all` re-includes intermediate anchors.)

Moment constraints on MFM path

At each constrained time $t_k \in \mathcal{T}_c$:

$$c_k(\theta) = \phi\Big(\{\mu_{t_k}^{(i)}(\theta)\}_{i=1}^{B}\Big) - m_k.$$

Define average squared residual

$$\overline{C}(\theta) = \frac{1}{|\mathcal{T}_c|} \sum_k \|c_k(\theta)\|_2^2.$$

Hybrid Stage A objectives

With coupling weight $\eta_{\text{moment}} \geq 0$:

Soft variant (`metric_constrained_soft`)

$$\mathcal{L}_A^{\text{soft}}(\theta) = \mathcal{L}_{\text{LAND}}(\theta) \;+\; \eta_{\text{moment}}\, \overline{C}(\theta).$$

Augmented-Lagrangian variant (`metric_constrained_al`)

$$\mathcal{L}_A^{\text{AL}}(\theta, \lambda) = \mathcal{L}_{\text{LAND}}(\theta) \;+\; \eta_{\text{moment}} \left[\sum_k \lambda_k^\top c_k(\theta) \frac{\rho}{2} \sum_k \|c_k(\theta)\|_2^2 \right].$$

Multiplier update (code-equivalent, with clipping):

$$\lambda_k \leftarrow \text{clip}(\lambda_k + \rho\, c_k(\theta), -\lambda_{\max}, \lambda_{\max})\,.$$

Degenerate case check If $\eta_{\text{moment}} = 0$, both hybrid variants reduce to pure LAND Stage A:

$$\mathcal{L}_A = \mathcal{L}_{\text{LAND}}.$$

Fairness statement

Under the default `endpoints_only` policy:

- LAND sees only p_0 and p_1 samples;
- intermediate information enters only via $\{m_k\}_{t_k \in \mathcal{T}_c}$ through $c_k(\theta)$.

This enforces comparable intermediate-information access between constrained and metric-constrained methods.

Algorithm: Hybrid MFM Stage A/B (AL and Soft variants)

Inputs: $\mathcal{T}_c$, targets $\{m_k\}$, $(\rho, \lambda_{\max}, \eta_{\text{moment}})$, LAND params $(\gamma_{\text{land}}, \rho_{\text{land}})$, steps (N_A, N_B).

1. Build endpoint manifold pool $\mathcal{M}_{\text{end}}$ (default policy).
2. If AL variant, initialize $\lambda_k \leftarrow 0$ for all $t_k \in \mathcal{T}_c$.
3. **Stage A** for $s = 1, \ldots, N_A$:

(a) Sample paired endpoints $\{(x_0^{(i)}, x_1^{(i)})\}_{i=1}^B$ and $t^{(i)} \sim U[0,1]$.
(b) Compute $(\mu_{t^{(i)}}, u_{t^{(i)}})$ from MFM geopath equations.
(c) Compute $\mathcal{L}_{\text{LAND}}$ using $\mathcal{M}_{\text{end}}$.
(d) Compute $c_k(\theta)$ for all constrained times.
(e) If soft variant: optimize $\mathcal{L}_A^{\text{soft}}$.
(f) If AL variant: optimize $\mathcal{L}_A^{\text{AL}}$, then update all λ_k by clipped AL rule.

4. **Stage B** for $s = 1, \ldots, N_B$:
   (a) Freeze g_θ.
   (b) Sample (t, x_t, u_t) with metric backend:
   $$x_t = \mu_t + \sigma_t \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I).$$
   (c) Optimize velocity model with $\mathcal{L}_{\text{CFM}}^{\text{metric}}$.

9 Why This is More Complex than "Path First, then Flow Matching"

Your suggested two-step idea is:

1. Learn paths (e.g., with a smoothness/Lipschitz regularizer),
2. then train flow matching on those paths.

Current implementation adds two extra mechanisms:

- **Augmented Lagrangian** instead of fixed penalties, to push constraints more aggressively.
- **Stage C joint finetune**, to co-adapt path model and velocity model after separate training.

So, yes: the present loop is strictly more complicated than a plain two-stage approach. It can improve constraint handling, but it also introduces additional instability/tuning sensitivity.

10 How Reported Metrics Are Computed

This section defines the fields written in `metrics.json` and `comparison.json`.

Constraint metrics

For each constrained time $t_k \in \mathcal{T}_c$, compute residual vector

$$c_k = \phi\Big(\{x_{t_k}^{(i)}\}_{i=1}^B\Big) - m_k.$$

Then:

$$\texttt{constraint_residual_norms}[t_k] = \|c_k\|_2.$$

The summary scalar is:

$$\texttt{constraint_residual_avg} = \frac{1}{|\mathcal{T}_c|} \sum_{t_k \in \mathcal{T}_c} \|c_k\|_2.$$

Validation CFM metric

On an evaluation batch:

1. sample paired (x_0, x_1) via the configured coupling mode,
2. sample $t \sim \text{Uniform}(0, 1)$,
3. build (x_t, u_t) from current mode (baseline or constrained),
4. evaluate

$$\texttt{cfm_val_loss} = \frac{1}{B} \sum_{i=1}^{B} \left\| v_\psi(t_i, x_{t_i}^{(i)}) - u_{t_i}^{(i)} \right\|_2^2 .$$

Intermediate-time Wasserstein metric

For each constrained time $t_k \in \mathcal{T}_c$:

1. Sample $x_0^{(i)} \sim p_0$, $i = 1, \ldots, N$.
2. Integrate the learned velocity field from 0 to t_k with Euler to obtain $\hat{x}_{t_k}^{(i)}$.
3. Compute empirical generated moments:

$$\hat{\mu}_{t_k} = \frac{1}{N} \sum_{i=1}^{N} \hat{x}_{t_k}^{(i)}, \qquad \hat{\Sigma}_{t_k} = \frac{1}{N} \sum_{i=1}^{N} (\hat{x}_{t_k}^{(i)} - \hat{\mu}_{t_k})(\hat{x}_{t_k}^{(i)} - \hat{\mu}_{t_k})^\top .$$

4. Let target moments be $(\mu_{t_k}, \Sigma_{t_k})$ from the analytic Gaussian bridge.

The reported per-time metric uses Gaussian W_2 distance:

$$\texttt{intermediate_w2_gaussian}[t_k] = W_2\Big(\mathcal{N}(\hat{\mu}_{t_k}, \hat{\Sigma}_{t_k}), \mathcal{N}(\mu_{t_k}, \Sigma_{t_k})\Big) ,$$

with closed-form

$$W_2^2 = \|\hat{\mu}_{t_k} - \mu_{t_k}\|_2^2 + \text{Tr}\left(\hat{\Sigma}_{t_k} + \Sigma_{t_k} - 2 \left(\Sigma_{t_k}^{1/2} \hat{\Sigma}_{t_k} \Sigma_{t_k}^{1/2} \right)^{1/2} \right) .$$

The summary scalar is:

$$\texttt{intermediate_w2_gaussian_avg} = \frac{1}{|\mathcal{T}_c|} \sum_{t_k \in \mathcal{T}_c} \texttt{intermediate_w2_gaussian}[t_k].$$

Sample-based empirical OT variant To support non-Gaussian settings, we also report a sample-based empirical W_2 proxy:

1. Generate trajectory samples $\{\hat{x}_{t_k}^{(i)}\}_{i=1}^N$ by integrating v_ψ from 0 to t_k.
2. Draw target samples $\{y_{t_k}^{(j)}\}_{j=1}^N$ from the reference distribution at t_k.
3. Compute the optimal bipartite matching $\pi^\star$ minimizing squared Euclidean cost:

$$\pi^\star = \arg\min_{\pi \in S_N} \sum_{i=1}^{N} \left\| \hat{x}_{t_k}^{(i)} - y_{t_k}^{(\pi(i))} \right\|_2^2 .$$

The reported per-time empirical metric is:

$$\texttt{intermediate_empirical_w2}[t_k] = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left\|\hat{x}_{t_k}^{(i)} - y_{t_k}^{(\pi^\star(i))}\right\|_2^2}.$$

and the average is:

$$\texttt{intermediate_empirical_w2_avg} = \frac{1}{|\mathcal{T}_c|}\sum_{t_k\in\mathcal{T}_c}\texttt{intermediate_empirical_w2}[t_k].$$

In code this uses exact discrete OT matching (Hungarian algorithm), so evaluation sample count is configured separately to control CPU cost.

Implementation note (current code path) In practice, this empirical OT metric uses

$$N = \texttt{eval_intermediate_ot_samples},$$

with a current default of $N = 256$ in the single-cell train profiles. Each run computes this metric from one sampled evaluation set (rather than averaging over multiple independent evaluation batches inside the run). For each evaluated timepoint, Hungarian matching is solved on that sampled set at that time. Therefore `intermediate_empirical_w2_avg` is an average over timepoints, not an average over multiple evaluation batches.

Full-set balanced OT variant (robust, additive; single-cell) In addition to the legacy sampled Hungarian metrics above, single-cell evaluation can report a robust full-set balanced OT family using exact LP on possibly rectangular supports (unequal cardinalities allowed). Legacy fields are kept unchanged for backward comparability.

Given weighted empirical measures

$$\hat{\mu}_t = \sum_{i=1}^{n_t} a_i\,\delta_{x_t^{(i)}}, \qquad \nu_t = \sum_{j=1}^{m_t} b_j\,\delta_{y_t^{(j)}},$$

with $a_i, b_j \geq 0$, $\sum_i a_i = \sum_j b_j = 1$, define

$$W_2^2(\hat{\mu}_t, \nu_t) = \min_{\Gamma\in\mathbb{R}_+^{n_t\times m_t}} \sum_{i=1}^{n_t}\sum_{j=1}^{m_t}\Gamma_{ij}\,\|x_t^{(i)} - y_t^{(j)}\|_2^2$$

subject to

$$\sum_j \Gamma_{ij} = a_i, \qquad \sum_i \Gamma_{ij} = b_j.$$

Reported metric value is $W_2 = \sqrt{W_2^2}$.

Stage-A robust definition (weighted edge measure) With global endpoint plan support $\mathcal{E} = \{(i, j, \Pi_{ij})\}$ from `ot_global`:

$$\hat{\mu}_t^{\text{lin}} = \sum_{(i,j)\in\mathcal{E}} \Pi_{ij}\,\delta_{(1-t)x_0^{(i)} + t x_1^{(j)}},$$

$$\hat{\mu}_t^{\text{learned}} = \sum_{(i,j)\in\mathcal{E}} \Pi_{ij}\, \delta_{x_t^{(i,j)}(\theta)},$$

where $x_t^{(i,j)}(\theta)$ is the mode-specific learned interpolant (constrained-correction or MFM path). Target measure at time t is uniform over the full observed pool at that timestamp:

$$\nu_t = \frac{1}{|\mathcal{Y}_t|} \sum_{y\in\mathcal{Y}_t} \delta_y.$$

Reported keys are:

`linear_full_ot_w2`$[t]$, `learned_full_ot_w2`$[t]$, `linear_full_ot_w2_avg`, `learned_full_ot_w2_avg`,

plus holdout and delta analogs.

A+B rollout robust definition For velocity-enabled single-cell runs, push forward the full source endpoint pool $\{x_0^{(i)}\}_{i=1}^{N_0}$:

$$\hat{x}_t^{(i)} = \Phi_t^{v_\psi}(x_0^{(i)}),$$

then compare uniform generated and full observed target pools via balanced OT:

`intermediate_full_ot_w2`$[t_k]$, `intermediate_full_ot_w2_avg`, `transport_endpoint_full_ot_w2`, `holdout_`

These robust keys are additive and do not replace legacy sampled Hungarian fields.

Stage-A-only interpolant evaluation (no velocity rollout)

For Stage-A-only runs ($N_B = N_C = 0$), we intentionally evaluate only interpolants:

1. Sample endpoint pairs $(x_0^{(i)}, x_1^{(i)})$ using configured coupling (`ot`, `random`, or `ot_global`).
2. For each constrained time t_k:

$$x_{t_k,\text{lin}}^{(i)} = (1-t_k)x_0^{(i)} + t_k x_1^{(i)},$$

$$x_{t_k,\text{learned}}^{(i)} = (1-t_k)x_0^{(i)} + t_k x_1^{(i)} + t_k(1-t_k)g_\theta(t_k, x_0^{(i)}, x_1^{(i)}).$$

3. Draw reference bridge samples $\{y_{t_k}^{(j)}\}_{j=1}^{N}$ at the same t_k from cached target marginals.
4. Compute empirical OT distances for linear and learned interpolants:

$$\texttt{linear_empirical_w2}[t_k] = W_2^{\text{emp}}\big(\{x_{t_k,\text{lin}}^{(i)}\}, \{y_{t_k}^{(j)}\}\big),$$

$$\texttt{learned_empirical_w2}[t_k] = W_2^{\text{emp}}\big(\{x_{t_k,\text{learned}}^{(i)}\}, \{y_{t_k}^{(j)}\}\big).$$

Reported summary fields:

`linear_empirical_w2_avg`, `learned_empirical_w2_avg`, `delta_avg_learned_minus_linear` = `learned_avg−`

Implementation note (current code path) Stage-A-only evaluation draws one endpoint sample set

$$\{(x_0^{(i)}, x_1^{(i)})\}_{i=1}^{N}, \qquad N = \texttt{eval_intermediate_ot_samples}$$

once per run (current single-cell default: $N = 256$). The same sampled endpoints are reused to build linear and learned interpolants at all evaluation times. At each evaluation time, a target sample set of size N is drawn once, then Hungarian matching is solved separately for linear-vs-target and learned-vs-target sets. Consequently, `linear_empirical_w2_avg` and `learned_empirical_w2_avg` are averages over evaluation times only, not averages over multiple evaluation batches. In this profile, velocity-rollout metrics are out-of-scope and stored as `null` in JSON outputs (`cfm_val_loss`, transport metrics, and velocity-based intermediate metrics).

Bridge Stage-A+B rollout evaluation

For bridge runs with velocity training enabled (for example `train=ab_only`), evaluation uses rollout marginals from the learned velocity field:

1. Sample $x_0^{(i)}$ from the bridge start marginal pool.
2. Integrate v_ψ with Euler to obtain generated samples $\hat{x}_t^{(i)}$ at normalized times
$$t \in \{0.25, 0.50, 0.75, 1.00\}.$$
3. Draw true bridge reference samples $\{y_t^{(j)}\}_{j=1}^{N}$ at the same normalized times from the cached target sampler (internally mapped to physical time via `bridge.total_time`).
4. Compute empirical OT distances
$$\widehat{W}_2(t) = W_2^{\text{emp}}\big(\{\hat{x}_t^{(i)}\}, \{y_t^{(j)}\}\big).$$

Reported bridge rollout fields are:

$$\texttt{intermediate_empirical_w2}[t_k] = \widehat{W}_2(t_k), \quad \texttt{intermediate_empirical_w2_avg} = \frac{1}{3} \sum_{t_k \in \{0.25, 0.50, 0.75\}} \widehat{W}_2(t_k),$$

$$\texttt{transport_endpoint_empirical_w2} = \widehat{W}_2(1.00), \quad \texttt{transport_score} = \texttt{transport_endpoint_empirical_w2}.$$

For bridge velocity-enabled runs, Gaussian-only transport fields are intentionally `null` (`transport_mean_error_l2`, `transport_cov_error_fro`, `intermediate_w2_gaussian`, `intermediate_w2_gaussian_avg`).

Path energy proxy

Using the same evaluation batch/time samples as `cfm_val_loss`:

$$\texttt{path_energy_proxy} = \frac{1}{B} \sum_{i=1}^{B} \|u_{t_i}^{(i)}\|_2^2.$$

This is a diagnostic quantity, not an objective minimized directly at evaluation time.

Transport quality metrics (Gaussian family)

To measure endpoint transport induced by v_ψ:

1. Sample $x_0^{(i)} \sim p_0$, $i = 1, \ldots, N$.
2. Integrate forward with explicit Euler over n_{steps} steps:

$$x_{n+1}^{(i)} = x_n^{(i)} + \Delta t\, v_\psi(t_n, x_n^{(i)}), \quad \Delta t = \frac{1}{n_{\text{steps}}}, \quad t_n = n\Delta t.$$

3. Let $\hat{x}_1^{(i)} = x_{n_{\text{steps}}}^{(i)}$.
4. Compute empirical endpoint statistics:

$$\hat{\mu}_1 = \frac{1}{N}\sum_{i=1}^{N} \hat{x}_1^{(i)}, \qquad \hat{\Sigma}_1 = \frac{1}{N}\sum_{i=1}^{N} (\hat{x}_1^{(i)} - \hat{\mu}_1)(\hat{x}_1^{(i)} - \hat{\mu}_1)^\top.$$

Then:

$$\texttt{transport_mean_error_l2} = \|\hat{\mu}_1 - \mu_1\|_2,$$
$$\texttt{transport_cov_error_fro} = \|\hat{\Sigma}_1 - \Sigma_1\|_F,$$
$$\texttt{transport_score} = \texttt{transport_mean_error_l2} + \texttt{transport_cov_error_fro}.$$

Stage-history diagnostics

During training, the history log also records:

- `avg_residual_norm`: mini-batch average of $\|c_k\|_2$ over constrained times,
- `loss`: stage objective value,
- and stage-specific terms (e.g., regularizer or `cfm_loss` in Stage C).

11 Unsupervised Soft Pseudo-Type Constraints for Single-Cell Data

This section formalizes the optional unlabeled-cell extension where class-like constraints are learned from data rather than provided as annotations.

GMM posterior definition

Given whitened single-cell features $x \in \mathbb{R}^d$, define a K-component mixture

$$p_\phi(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k), \qquad \pi_k \geq 0,\ \sum_k \pi_k = 1.$$

The soft pseudo-type membership is

$$q_k(x) = P_\phi(z = k \mid x) = \frac{\pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x \mid \mu_j, \Sigma_j)}.$$

In code, (π_k, μ_k, Σ_k) are frozen after preprocessing and $q_k(x)$ is evaluated with a differentiable torch implementation during training.

K selection by BIC + stability

For each $K \in [K_{\min}, K_{\max}]$ and each seed $s \in \mathcal{S}$:

1. fit GMM $\phi_{K,s}$;
2. compute $\mathrm{BIC}_{K,s}$ on full feature matrix;
3. compute hard labels $\hat{z}_{K,s}(x) = \arg\max_k q_k(x; \phi_{K,s})$.

Define

$$\widetilde{\mathrm{BIC}}_K = \mathrm{median}_{s\in\mathcal{S}} \mathrm{BIC}_{K,s},$$

$$\mathrm{Stab}_K = \frac{2}{|\mathcal{S}|(|\mathcal{S}|-1)} \sum_{s_1<s_2} \mathrm{ARI}(\hat{z}_{K,s_1}, \hat{z}_{K,s_2}) \,.$$

Given threshold τ, let

$$\mathcal{K}_\tau = \{K : \mathrm{Stab}_K \geq \tau\}.$$

Selection rule:

$$K^\star = \begin{cases} \arg\min_{K\in\mathcal{K}_\tau} \widetilde{\mathrm{BIC}}_K, & \mathcal{K}_\tau \neq \varnothing, \\ \arg\min_{K\in[K_{\min},K_{\max}]} \widetilde{\mathrm{BIC}}_K, & \text{otherwise.} \end{cases}$$

Diagnostics reported in summaries are `bic_by_k` and `stability_by_k`.

Pseudo-targets and residuals

For each constrained/observed time t with dataset pool $\mathcal{D}_t = \{x_t^{(n)}\}_{n=1}^{N_t}$:

$$\pi_{t,k}^{\mathrm{ps}} = \frac{1}{N_t} \sum_{n=1}^{N_t} q_k(x_t^{(n)}).$$

For generated samples $\{\hat{x}_t^{(i)}(\theta)\}_{i=1}^{B}$:

$$\hat{\pi}_{t,k}^{\mathrm{ps}}(\theta) = \frac{1}{B} \sum_{i=1}^{B} q_k\Big(\hat{x}_t^{(i)}(\theta)\Big) \,.$$

Pseudo residual vector at time t:

$$r_t^{\mathrm{ps}}(\theta) = \hat{\pi}_t^{\mathrm{ps}}(\theta) - \pi_t^{\mathrm{ps}}.$$

Hybrid constrained objective (moments + pseudo)

Constrained mode objective becomes

$$\mathcal{L}_{\mathrm{constr}} = R(\theta) + \mathcal{L}_{\mathrm{AL,mom}}(\theta, \lambda) + \eta_{\mathrm{ps}}\, \mathcal{L}_{\mathrm{AL,ps}}(\theta, \nu).$$

with

$$\mathcal{L}_{\mathrm{AL,ps}}(\theta, \nu) = \sum_{t\in\mathcal{T}_c} \nu_t^\top r_t^{\mathrm{ps}}(\theta) + \frac{\rho_{\mathrm{ps}}}{2} \sum_{t\in\mathcal{T}_c} \|r_t^{\mathrm{ps}}(\theta)\|_2^2.$$

Multiplier update:

$$\nu_t \leftarrow \mathrm{clip}(\nu_t + \rho_{\mathrm{ps}}\, r_t^{\mathrm{ps}}(\theta), -\lambda_{\max}^{\mathrm{ps}}, \lambda_{\max}^{\mathrm{ps}}) \,.$$

Backward compatibility: default `pseudo_eta`=0.0 disables pseudo contribution.

Metric-constrained variants with pseudo terms

AL variant (`metric_constrained_al`)

$$\mathcal{L}_A^{\text{metric_AL+ps}} = \mathcal{L}_{\text{LAND}} + \eta_{\text{moment}}\mathcal{L}_{\text{AL,mom}} + \eta_{\text{ps}}\mathcal{L}_{\text{AL,ps}}.$$

Soft variant (`metric_constrained_soft`)

$$\mathcal{L}_A^{\text{metric_soft+ps}} = \mathcal{L}_{\text{LAND}} + \eta_{\text{moment}}\frac{1}{|\mathcal{T}_c|}\sum_t \|c_t(\theta)\|_2^2 + \eta_{\text{ps}}\frac{1}{|\mathcal{T}_c|}\sum_t \|r_t^{\text{ps}}(\theta)\|_2^2.$$

Algorithm order (per training step)

Preprocessing (once/run):

1. fit/select/load GMM by BIC+stability;
2. freeze posterior $q_k(\cdot)$;
3. compute π_t^{ps} on observed constrained-time pools.

Stage A / Stage C constrained update:

1. sample endpoint pairs and times;
2. compute path samples at each constrained time;
3. compute moment residuals $c_t(\theta)$ and pseudo residuals $r_t^{\text{ps}}(\theta)$;
4. assemble total objective (regularizer + moment term + pseudo term);
5. gradient step on trainable parameters;
6. update multipliers:

$$\lambda_t \leftarrow \text{clip}(\lambda_t + \rho\, c_t,\, -\lambda_{\max}, \lambda_{\max}), \quad \nu_t \leftarrow \text{clip}(\nu_t + \rho_{\text{ps}}\, r_t^{\text{ps}},\, -\lambda_{\max}^{\text{ps}}, \lambda_{\max}^{\text{ps}}).$$

The same pseudo residual definitions are used for constrained and metric-constrained families.

12 Example Experiment Settings (Current Repository)

This section summarizes the experiment families currently implemented and logged in this repository.

A. Primary trained benchmark: 2D Gaussian → 2D Gaussian

- **Data family**: both endpoints are Gaussians in $\mathbb{R}^2$.
- **Targets**: intermediate moment targets are analytic Gaussian bridge moments at $t \in \{0.25, 0.50, 0.75\}$.
- **Pair coupling options**: exact discrete OT (`data=gaussian_ot`) or random permutation pairing (`data=gaussian_random`).

- **Training modes**: baseline (linear path), constrained (Stage A/B/C), and comparison (run both).
- **Typical profiles**: `train=default`, `train=ab_only` (Stage C disabled), and `train=smoke`.
- **Representative output roots**: `outputs/hparam_sweep/`, `outputs/ab_only_sweep/`, and timestamped comparison folders under `outputs/<date>/...`.

B. Bridge-SDE benchmark and preview utilities

- **Purpose**: non-Gaussian-style benchmark family with bridge geometry, plus notebook diagnostics.
- **Initial law**: Gaussian $p_0 = \mathcal{N}(\mu_0, \Sigma_0)$ from bridge-preview parameters.
- **Simulated dynamics** (from `bridge_sde.py`):

$$dX_t = v_x\, dt + \sigma_x\, dW_t^{(x)},$$

$$dY_t = -\kappa_b\, g(X_t)\, Y_t\, dt + \sigma_y(1 - \delta_b\, g(X_t))\, dW_t^{(y)}, \qquad g(x) = \exp\left(-\frac{(x-c)^2}{2w^2}\right).$$

- **Current status**: integrated in training for Stage-A-only interpolant studies and Stage-A+B velocity-rollout comparisons (Stage C still not used in this bridge milestone).
- **Training data configs**: `data=bridge_ot` and `data=bridge_random`.
- **Typical training profile**: `train=stage_a_only` (interpolant-only) or `train=ab_only` (A+B rollout metrics).
- **Output roots**: training artifacts under `outputs/<date>/...`; preview utilities under `outputs/preview_bri`

Scope note The core optimization in Sections 1–9 is shared across Gaussian and bridge families. What differs by family is the target construction and evaluation contract (Gaussian moment-based transport metrics vs bridge empirical rollout OT metrics).

13 Practical Hyperparameters in the Loop

Symbol / Config	Role
α	weight of velocity-deviation regularizer
β	weight of temporal smoothness term
ρ	augmented-Lagrangian quadratic and multiplier update scale
η_{joint}	weight of $\mathcal{L}_g$ inside Stage C objective
lr_g, lr_v	learning rates for path and velocity networks
N_A, N_B, N_C	number of steps in each stage